

FORMAL LANGUAGES AND COMPILERS

prof.s Luca Breveglieri and Angelo Morzenti

Exam of Wed 13 July 2016 - Part Theory

WITH SOLUTIONS - FOR TEACHING PURPOSES HERE THE SOLUTIONS ARE WIDELY
COMMENTED

NAME (capital letters pls.):

MATRICOLA:

SIGNATURE:

INSTRUCTIONS - READ CAREFULLY:

- The exam is in written form and consists of two parts:
 1. Theory (80%): Syntax and Semantics of Languages
 - regular expressions and finite automata
 - free grammars and pushdown automata
 - syntax analysis and parsing methodologies
 - language translation and semantic analysis
 2. Lab (20%): Compiler Design by Flex and Bison
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- To pass part theory, the candidate must answer the mandatory (not optional) questions; notice that the full grade is achieved by answering the optional questions.
- The exam is open book: textbooks and personal notes are permitted.
- Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.
- Time: part lab 60m - part theory 2h.15m

1 Regular Expressions and Finite Automata 20%

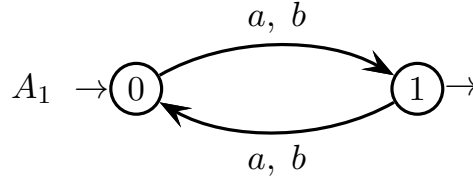
1. Consider the two-letter alphabet $\Sigma = \{ a, b \}$. Answer the following questions:
 - (a) Design a (preferably minimal) finite-state automaton A_1 that accepts all and only the strings of odd length over the alphabet Σ .
 - (b) By using the Berry-Sethi method (*BS*), find a deterministic finite-state automaton A_2 equivalent to the regular expression R below:

$$R = (a \mid b)^* b$$

- (c) In a systematic way, obtain the finite-state automaton A cartesian product of the previous automata A_1 and A_2 , i.e., $A = A_1 \times A_2$.
 - (d) If necessary, minimize the automaton A previously obtained.
 - (e) (optional) From the minimal automaton A and using the *BMC* method (node elimination method), derive an equivalent regular expression R' . If necessary, rewrite this regular expression R' in order to simplify it and to highlight the characteristic properties of the language generated by R' .
-

Solution

- (a) Here is the (deterministic) minimal automaton A_1 , obtained informally, that accepts only the strings of odd length over the two-letter alphabet Σ :

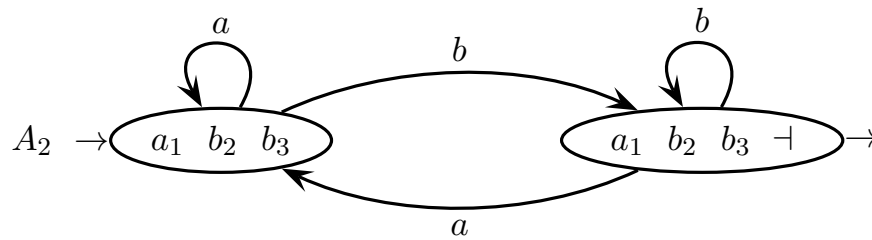


Automaton A_1 is correct: it accepts the two one-letter strings a and b as well as any other string of $1 + 2 = 3$, $3 + 2 = 5$, etc, letters over alphabet Σ , i.e., it accepts all the strings of odd length; while it does not accept the empty string ε (as the initial state is not final) or any other string of even length. Automaton A_1 is deterministic, in clean form and clearly minimal, as it has only two states, one non-final and one final, which are obviously distinguishable.

- (b) Here is the deterministic automaton A_2 obtained by means of the *BS* method. First the marked regular expression $R_{\#}$, and its two sets of initials and followers:

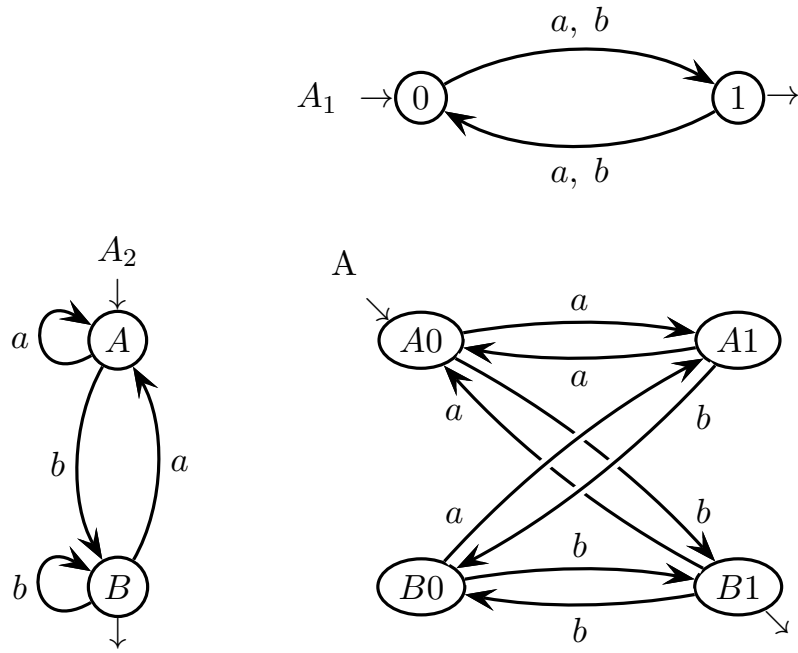
$R_{\#} = (a_1 \mid b_2)^* b_3 \dashv$	initials	$a_1 \quad b_2 \quad b_3$
	terminals	followers
	a_1	$a_1 \quad b_2 \quad b_3$
	b_2	$a_1 \quad b_2 \quad b_3$
	b_3	$\dashv$

Then the *BS* automaton A_2 that recognizes language $L(R)$:



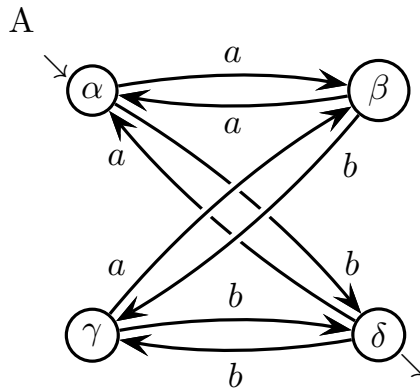
By construction, automaton A_2 is deterministic and in clean form. It happens to be minimal, too, for the same reason as for automaton A_1 .

- (c) Here is the cartesian product automaton $A = A_1 \times A_2$ that accepts the intersection of the two languages $L(A_1)$ and $L(A_2)$:

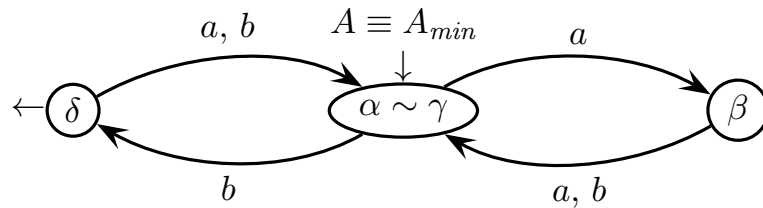


Since both automata A_1 and A_2 are deterministic, their product automaton A is deterministic as well. Here it also happens that automaton A is in clean form (this is not a necessary property). For minimality, see the next question.

- (d) Since automaton A is deterministic, we can check if it is minimal and if necessary minimize it. First we make a change of state names, to simplify the notation:

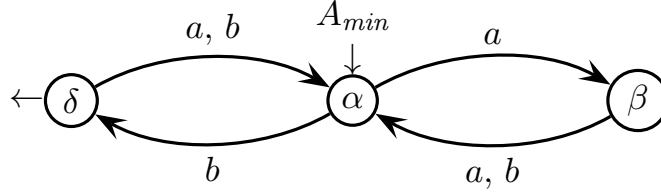


We immediately see that states α and γ are undistinguishable, as both are non-final, have the same outgoing letters, and go to the same states β and δ by letters a and b , respectively. Here is the result of merging states α and γ :

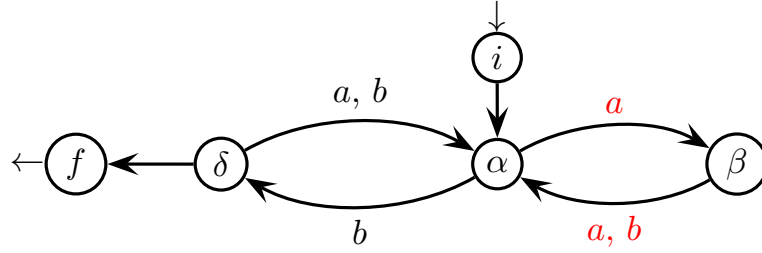


Automaton A_{min} is in minimal form: state δ is final and thus it is distinguishable from the other two non-final states; the merged state $\alpha \sim \gamma$ and state β are distinguishable as the former goes to a final state while the latter does not.

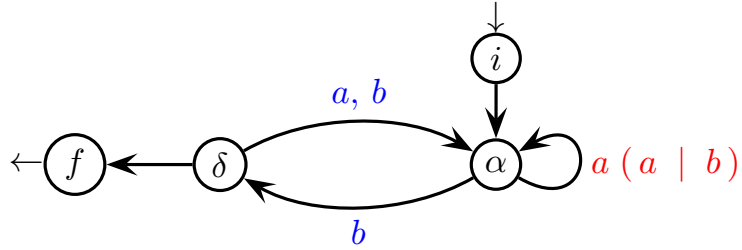
(e) We rename state $\alpha \sim \gamma$ as α , then we proceed by node elimination (Brzozowski):



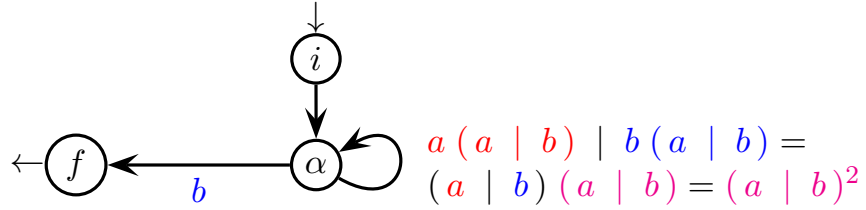
Introduce the unique non-reentrant initial and final states i and f , respectively (from now on the coloring will help follow each node elimination step):



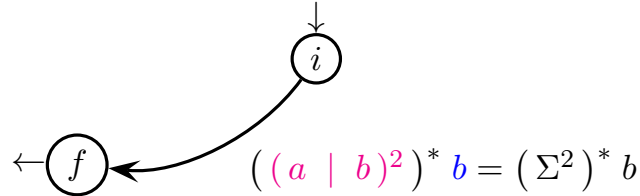
Eliminate node β :



Eliminate node δ (magenta highlights a common factoring of red and blue):



Eliminate node α :



The final regular expression $R' = ((a \mid b)^2)^* b$ shows that the language $L(R') = L(A)$ of the product automaton $A = A_1 \times A_2$ consists of all and only the strings of odd length ending with a letter b . Intuitively, this is equivalent to conjoining the logical features of languages $L(A_1)$, which consists of odd length strings, and $L(A_2)$, which consists of any strings terminated by a letter b .

2 Free Grammars and Pushdown Automata 20%

1. Consider the Dyck language with round open and closed parentheses ‘(’ and ‘)’, generated by the well known *BNF* unambiguous grammar G below (axiom S):

$$G: S \rightarrow (S) S \mid \varepsilon$$

Answer the following questions:

- (a) Write a grammar G_1 , *BNF* and unambiguous, that generates all and only the Dyck strings of language $L(G)$ whose substrings of consecutive open parentheses have an even length (0, 2, 4, ...).

Examples:

$$\varepsilon \qquad \overbrace{(())}^2 \qquad \overbrace{(())}^2 \overbrace{(())}^2$$

Counterexamples:

$$\overbrace{()}^1 \qquad \overbrace{(())}^2 \overbrace{((()))}^3$$

To test grammar G_1 , draw the syntax trees of the three above examples; also try to draw the syntax trees of the two counterexamples and shortly explain why it is impossible to complete the trees by means of grammar G_1 .

- (b) (optional) Write a grammar G_2 , *BNF* and unambiguous, that generates all and only the Dyck strings of language $L(G)$ whose substrings of consecutive closed parentheses have an odd length (1, 3, 5, ...).

Examples:

$$\overbrace{()}^1 \qquad \overbrace{(())}^1 \overbrace{((()))}^3$$

Counterexamples:

$$\varepsilon \qquad \overbrace{(())}^2 \qquad \overbrace{()}^1 \overbrace{(())}^2$$

To test grammar G_2 , draw the syntax trees of the two above examples; also try to draw the syntax trees of the three counterexamples and shortly explain why it is impossible to complete the trees by means of grammar G_2 .

Solution

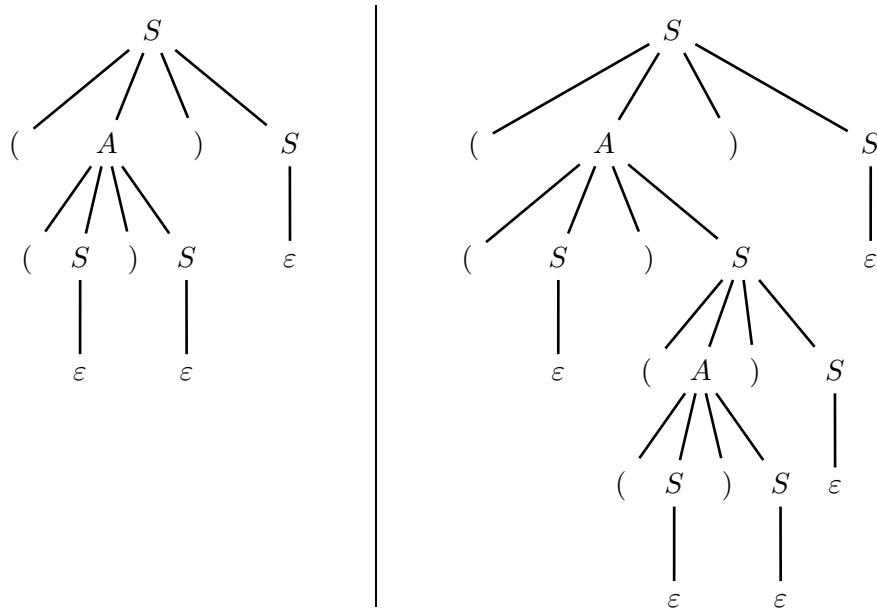
(a) Here is grammar G_1 (axiom S), inspired to grammar G :

$$G_1 \begin{cases} S \rightarrow (A)S \mid \varepsilon \\ A \rightarrow (S)S \end{cases}$$

Notice that grammar G_1 is right-recursive. Intuitively, grammar G_1 forces the consecutive open parentheses to come two-by-two (starting from zero). Since grammar G_1 is obtained by restricting the unambiguous grammar G , it is unambiguous as well.

The syntax trees of the examples are easy to draw, and those of the counterexamples cannot be completed as nonterminal A may not generate the empty string and thus prevents the tree from having an odd number of consecutive open parentheses on the frontier.

The tree of the valid empty string ε is trivial (use the axiomatic null rule $S \rightarrow \varepsilon$) and is skipped. Here are the two significant valid sample trees for grammar G_1 :



One can see that on every path from root to ε the nested nonterminals S and A alternate, but only nonterminal S is nullable, thus they can generate substrings of open parentheses only of even length.

As for the two invalid sample strings, their most fitting tentative syntax trees (yet incomplete) are as below:

This case uses both alternatives $S \rightarrow ((S)S)S$, twice, and $S \rightarrow \varepsilon$, five times. The reader may wish to try an invalid case by himself and see where the tree is unavoidably left incomplete.

Grammar G'_1 is compact and elegant, but it requires to generalize the standard Dyck rule. Of course, other different solutions may still exist.

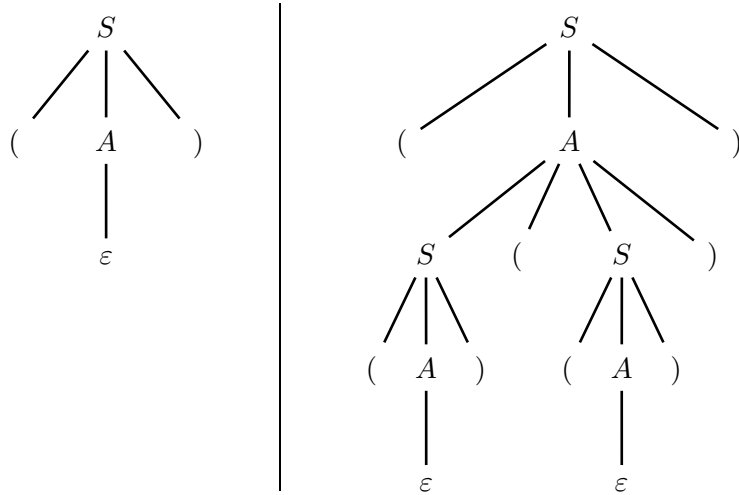
- (b) Here is grammar G_2 (axiom S), also inspired to grammar G , but with some more relevant changes with respect to grammar G_1 :

$$G_2 \begin{cases} S \rightarrow S(A) \mid (A) \\ A \rightarrow S(S) \mid (S) \mid \varepsilon \end{cases}$$

Notice that grammar G_2 is left-recursive; in fact, it is well known that grammar G can be equivalently written in left-recursive form, i.e., “ $S \rightarrow S(S) \mid \varepsilon$ ”. This change allows us to easily control the number of closed parentheses, instead of the open ones. Intuitively, grammar G_2 forces the consecutive closed parentheses to come two-by-two, plus one more closed parenthesis as nonterminal S is unable to generate the empty string ε . Notice also that the non-nullability of nonterminal S makes it necessary to introduce a few alternatives, to model the cases where there are not any concatenated parenthesis pairs, but only nested pairs. Since grammar G_2 is obtained by restricting the unambiguous left-recursive version of grammar G , it is unambiguous as well.

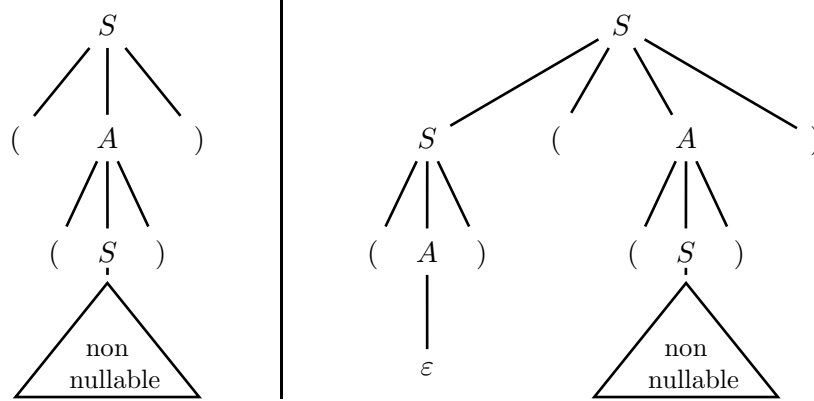
The syntax trees of the examples are easy to draw, and those of the counterexamples cannot be completed as nonterminal S may not generate the empty string and thus prevents the tree from having an even number of consecutive closed parentheses on the frontier.

Here are the two valid sample trees for grammar G_2 :



One can see that on every path from root to ε the nested nonterminals S and A alternate, but only nonterminal A is nullable, thus they can generate substrings of closed parentheses only of odd length.

Clearly the invalid empty string ε cannot be generated (the axiom S is evidently non-nullable). As for the two significant invalid sample strings, their most fitting tentative syntax trees (yet incomplete) are as below:



Other larger tentative trees would fail even worse. In both cases, for having the desired frontier one should nullify nonterminal S , which is not nullable. This happens wherever an even number of consecutive closed parentheses has been placed on the frontier, that is, it has been generated by grammar G_2 .

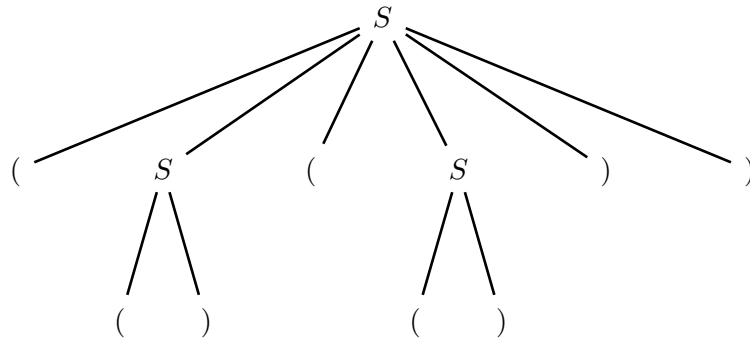
Similarly to question (a), a different solution consists of forcing the Dyck rule to always generate two closed parentheses at a time, plus one more at the derivation end, so as to always have an odd number thereof. Of course, it is again necessary to move recursion onto the left. Here it is, named grammar G'_2 (axiom S):

$$G'_2 \left\{ \begin{array}{l} S \rightarrow \textcolor{red}{S}(\textcolor{red}{S}(\textcolor{red}{S})) \mid \textcolor{blue}{S}() \\ S \rightarrow (\textcolor{red}{S}(\textcolor{red}{S})) \mid \textcolor{red}{S}((\textcolor{red}{S})) \mid ((\textcolor{red}{S})) \mid (()) \end{array} \right.$$

The main rule is the top one. In the first alternative (red), the two closed parentheses match the corresponding open ones, which may have a Dyck substring concatenated on the left. This requires concatenating a nonterminal S on the left of each open parenthesis. Similarly for the second alternative (blue), which instead models the case where there is only one more closed parenthesis to make an odd number. The meaning of the coloring is clarified below.

The reader may notice that a terminal rule is still missing. The bottom rule contains four more alternatives (unnecessary for question (a)), to take into account that now (differently from question (a)) nonterminal S is non-nullable, though an open parenthesis does not necessarily need to have a Dyck substring concatenated on the left. Thus in the case there is not any concatenation, each occurrence of S on the left side of an open parenthesis in the first top alternative (red) has to be canceled independently of the other, and this gives rise to three cases (also red). The reader may notice that such a reasoning is analogous to the algorithm for putting a grammar into non-nullable normal form. Similarly for the second top alternative, but this gives rise to only one case (blue) and furthermore provides the still missing yet indispensable terminal rule. The coloring helps us relate the alternatives in the two rules.

The reader may test grammar G'_2 by himself, using the same examples and counterexamples as provided before. Just to show one valid case:



This case uses the alternatives $S \rightarrow (S (S))$, once, and $S \rightarrow ()$, twice. The reader may wish to try an invalid case by himself and see where the tree is unavoidably left incomplete.

Grammar G'_2 also generalizes the standard Dyck rule, but it is somewhat more complicated and less elegant than grammar G'_1 , although both grammars are inspired to the same principle. In fact, for G'_2 the requirement of having an odd number of consecutive closed parentheses prevents us from using the empty string ε , which restriction gives rise to a few alternatives. Of course, other different solutions may still exist.

2. Consider a language designed to describe a sport game organized by elimination tournaments, where at each stage of the game only the winners will play in the next stage, until exactly one competitor becomes the final winner.

- In each stage, a player can be either a new team or the winner team of a previous stage, at any depth level.
- Every match has two players and these attributes:
 - an associated date, defined by a number for the year, a string for the month and a number for the day
 - a scheduled time, defined by two numbers for the hour and the minute
 - and a location, defined by a string

For simplicity, assume that in the grammar all the numbers and strings are schematized by the terminals *n* and *s*, respectively.

- A document (i.e., a string of the language) consists of a non-empty list of tournament descriptions.

Answer the following questions:

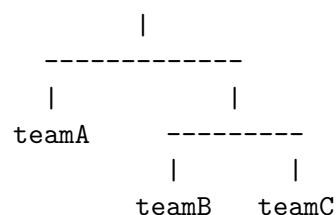
- Write a grammar *G*, *EBNF* and unambiguous, that models the language described above.
- Briefly illustrate how your grammar *G* can be modified in a systematic way to put a limit *k* on the height of the tree that represents the matches played in the tournament; exemplify the modification for the case *k* = 3.

Two sample language fragments are reported below. On the right side of each sample it is depicted a tree that represents the matches of the tournament.

```

tournament julyFun is
  match
    date 2016 july 29;
    time 21 : 30;
    location Rome;
    player1 is teamA;
    player2 is winner of match
      date 2016 july 23;
      time 17 : 00;
      location Milan;
      player1 is teamB;
      player2 is teamC;
    end match;
  end match;
end tournament julyFun;

```

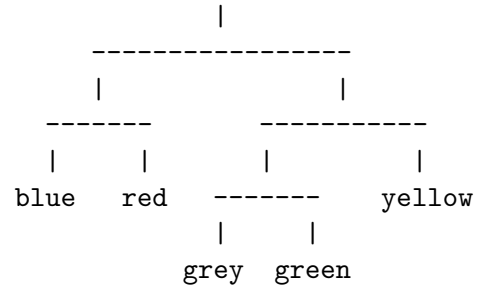


See the next page for one more example.

```

tournament springOutdoor is
  match
    ...
    player1 is winner of match
    ...
    player1 is blue
    player2 is red;
  end match;
  player2 is winner of match
  ...
  player1 is winner of match
  ...
  player1 is grey;
  player2 is green;
  end match;
  player2 is yellow;
  end match;
end match;
end tournament springOutdoor;

```



Solution

- (a) Here is a reasonable grammar G for the sketched language (axiom **DOCUM**):

$$G \left\{ \begin{array}{l} \langle \text{DOCUM} \rangle \rightarrow (\langle \text{TOURN} \rangle \text{ ' ; ' })^+ \\ \hline \langle \text{TOURN} \rangle \rightarrow \text{tournament } s \text{ is} \\ \qquad \qquad \qquad \langle \text{MATCH} \rangle \text{ ' ; ' } \\ \qquad \qquad \qquad \text{end tournament } s \\ \hline \langle \text{MATCH} \rangle \rightarrow \text{match} \\ \qquad \qquad \qquad \langle \text{DATE} \rangle \text{ ' ; ' } \\ \qquad \qquad \qquad \langle \text{TIME} \rangle \text{ ' ; ' } \\ \qquad \qquad \qquad \langle \text{LOCATION} \rangle \text{ ' ; ' } \\ \qquad \qquad \qquad \langle \text{PLAYER1} \rangle \text{ ' ; ' } \\ \qquad \qquad \qquad \langle \text{PLAYER2} \rangle \text{ ' ; ' } \\ \qquad \qquad \qquad \text{end match} \\ \hline \langle \text{PLAYER1} \rangle \rightarrow \text{player1 is } (s \mid \text{winner of } \langle \text{MATCH} \rangle) \\ \langle \text{PLAYER2} \rangle \rightarrow \text{player2 is } (s \mid \text{winner of } \langle \text{MATCH} \rangle) \\ \langle \text{DATE} \rangle \rightarrow \text{date } n \text{ } s \text{ } n \\ \langle \text{TIME} \rangle \rightarrow \text{time } n \text{ ' : ' } n \\ \langle \text{LOCATION} \rangle \rightarrow \text{location } s \end{array} \right.$$

Grammar G is of type *EBNF* and likewise is unambiguous. Most of G is made of well known unambiguous constructs: regular lists with separators and disjoint alternatives. Recursion involves only the nonterminal **MATCH**, passing through the nonterminals **PLAYER x** (with $x = 1, 2$), and it does not cause any ambiguity either. In fact, a **MATCH** instance may recursively generate only up to two different (inner) **MATCH** instances, but each of them is orderly preceded by the terminal **player x** (with $x = 1, 2$), so that they are distinguishable from each other and thus cannot give rise to ambiguity. There are not any other constructs that might cause ambiguity, so grammar G is unambiguous by construction.

Notice that the semicolon separators are mostly grouped in the **MATCH** rule, that is, they are modeled as a part of the match declaration frame. In this way, the separators can be manipulated homogeneously. Alternatively, one might wish to specify each of them in the rule of each list element, i.e., at the end of rules **DATE**, **TIME**, **LOCATION**, etc. Though the choice is essentially a matter of style, this alternative is less handy if some changes have to be made subsequently. Of course, there may still exist different solutions.

- (b) A simple solution to bound the depth of the competition tree, consists of introducing distinct nonterminals for the matches that take place at different tree levels. For the case $k = 3$, we will have three such nonterminals **MATCH1**, **MATCH2** and **MATCH3**, where the last one can be expanded only as a player name, not as the winner of a further match.

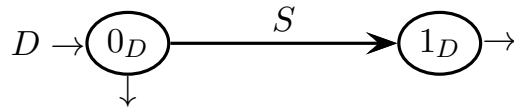
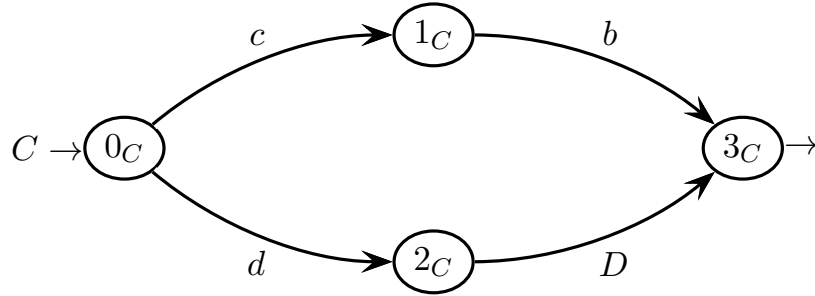
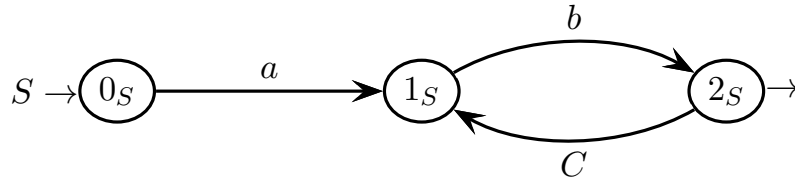
Below there is a sketch of the modified grammar (only the new or updated rules, or part thereof). To limit the grammar size, the expansions of nonterminals **PLAYER1** and **PLAYER2** are incorporated into the **MATCHy** rules (with $y = 1, 2, 3$), and the nonterminals **PLAYER1** and **PLAYER2** are eliminated.

$$\text{modified grammar } G \left\{ \begin{array}{l}
 \langle \text{TOURN} \rangle \rightarrow \text{tournament } s \text{ is} \\
 \qquad \qquad \qquad \langle \text{MATCH1} \rangle \text{ ' ; ' } \\
 \qquad \qquad \qquad \text{end tournament } s \\
 \hline
 \langle \text{MATCH1} \rangle \rightarrow \text{match} \\
 \qquad \qquad \qquad \dots \\
 \qquad \qquad \qquad \text{player1 is } (s \mid \text{winner of } \langle \text{MATCH2} \rangle) \text{ ' ; ' } \\
 \qquad \qquad \qquad \dots \\
 \qquad \qquad \qquad \text{end match} \\
 \hline
 \langle \text{MATCH2} \rangle \rightarrow \text{match} \\
 \qquad \qquad \qquad \dots \\
 \qquad \qquad \qquad \text{player1 is } (s \mid \text{winner of } \langle \text{MATCH3} \rangle) \text{ ' ; ' } \\
 \qquad \qquad \qquad \dots \\
 \qquad \qquad \qquad \text{end match} \\
 \hline
 \langle \text{MATCH3} \rangle \rightarrow \text{match } \dots \text{ player1 is } s \text{ ' ; ' } \dots \text{ end match}
 \end{array} \right.$$

Here we still keep to the same choice as before, that of grouping the semicolon separators in the **MATCHy** rules (with $y = 1, 2, 3$), i.e., in the match declaration frames (thus modeling a match as a list with separators). Of course, there may still exist different solutions.

3 Syntax Analysis and Parsing Methodologies 20%

1. Consider the following grammar G , represented as a machine net over the four-letter terminal alphabet $\Sigma = \{ a, b, c, d \}$ and the three-letter nonterminal alphabet $V = \{ S, C, D \}$ (axiom S).

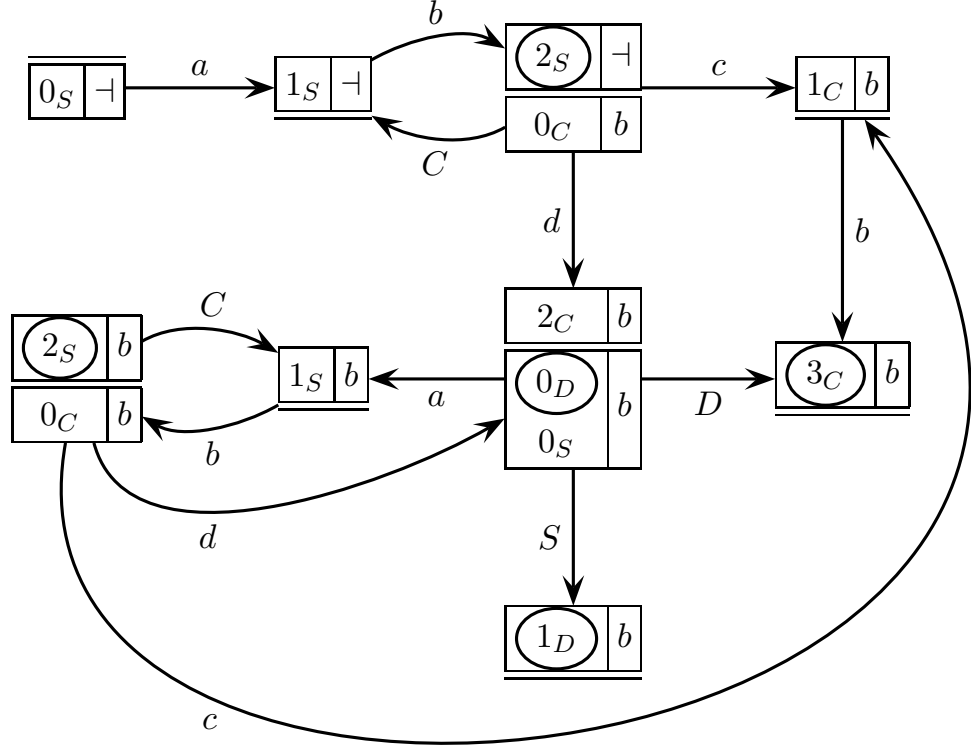


Answer the following questions:

- (a) Draw the complete pilot of grammar G , say if grammar G is of type $ELR(1)$ and shortly justify your answer.
- (b) Write all the guide sets on the arcs of the machine net (shift and call arcs, and final arrows), say if grammar G is of type $ELL(1)$, based on the guide sets, and shortly justify your answer.
- (c) (optional) Write two (of three) syntactic procedures of the recursive descent syntax analyzer of grammar G .

Solution

- (a) Here is the requested pilot, with 9 m-states, where for simplicity the m-state numbering is omitted and item $\langle 0_S, \neg \rangle$ is the initial m-state. The pilot does not have any conflicts, thus grammar G is of type $ELR(1)$.

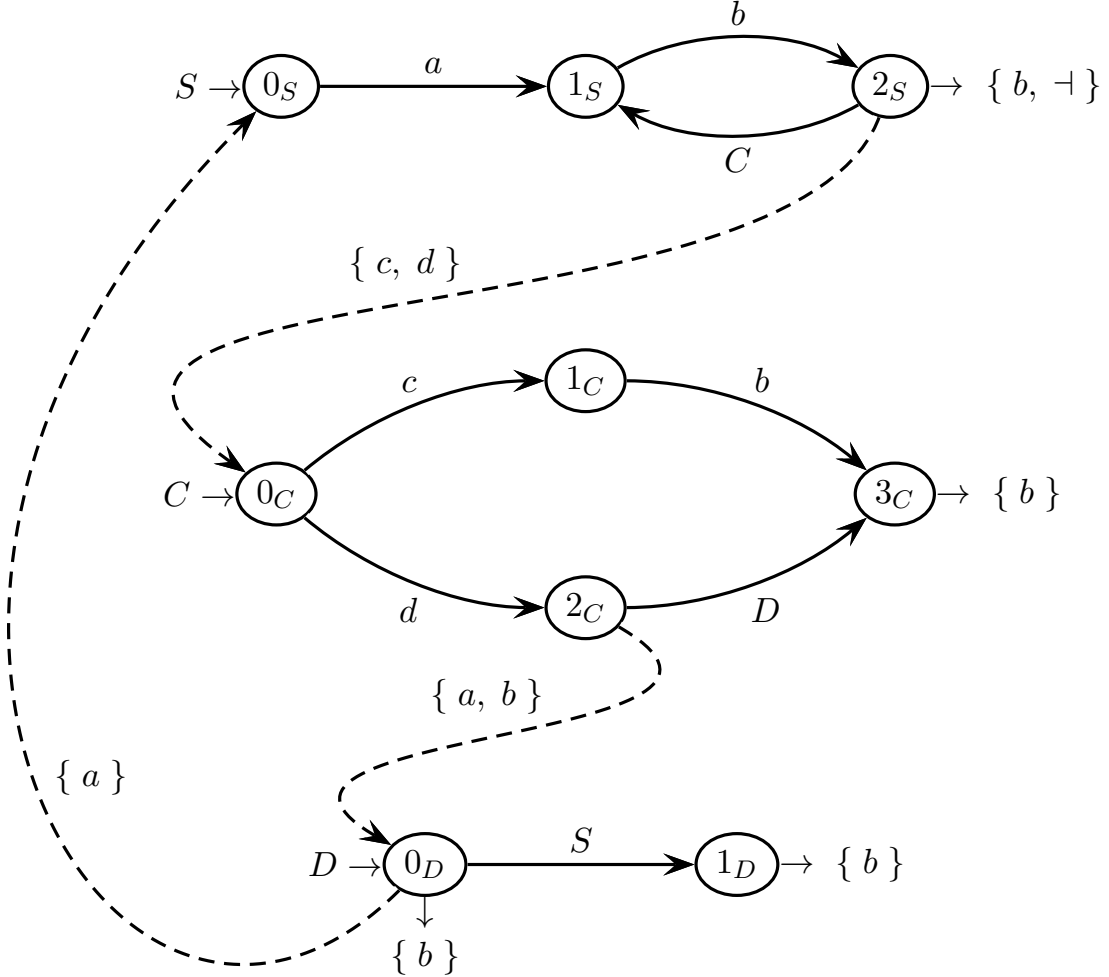


Incidentally, one may easily notice that the pilot has the Single Transition Property (STP), as all the m-state bases are singletons, which implies the STP . Furthermore, grammar G is not left-recursive, as the only left call chain is that nonterminal D calls the axiom S , but the chain is not recursive.

Overall, grammar G (or more precisely its pilot) is ELR , has the STP and is not left-recursive. Thus the ELL condition is satisfied and one straightforwardly concludes that grammar G is of type $ELL(1)$ as well.

To complete the picture, one might wonder whether language $L(G)$ is regular. Notice that grammar G has an auto-inclusive (self-embedding) recursive derivation scheme $S \Rightarrow a b C b \Rightarrow a b d D b \Rightarrow a b d S b$, which terminates either with $C \Rightarrow c b$ or with $D \Rightarrow \varepsilon$ or with $S \Rightarrow a b$. This is a necessary condition for a free language to be non-regular, but generally it is not sufficient. Anyway grammar G does not have any other derivation schemes. Therefore, by iterating the above scheme $S \Rightarrow a b C b \Rightarrow a b d D b \Rightarrow a b d S b$ and terminating it with the alternative $S \Rightarrow a b$, grammar G generates the non-regular string (sub)set $(a b d)^n a b b^n$ for any $n \geq 0$, with two equal exponents. All the other strings generated by G are just variations thereof, depending on which rule of G the derivation terminates by. Thus one can reasonably conclude that language $L(G)$ is unlikely to be regular.

- (b) Here are all the guide sets of the machine net, which are not difficult to compute. The machine net completed in this way constitutes the so-called *PCFG* (Predictive Control Flow Graph) of the *ELL* syntax analyzer.



Since the pilot has already been drawn and is available, the prospect sets (also called guide sets) on the exit arrows of the final states in the net can be obtained by collecting the look-ahead terminals on the reduction items of the m-states in the pilot. In this way the reader may verify the coherence of *PCFG* and pilot.

The guide set on call arc $2_C \xrightarrow{\{a, b\}} 0_D$ contains two elements, namely: letter a as it holds $a \in \text{Ini}(L(0_S))$ or equivalently the call arc is followed by call arc $0_D \xrightarrow{\{a\}} 0_S$; and letter b as nonterminal D is nullable, language $L(3_C)$ is also nullable and it holds $b \in \pi_{3_C}$, i.e., b is in the prospect set π of state 3_C . **Caution:** the reason for b to be in the guide set of $2_C \rightarrow 0_D$, is *not* that the call arc is followed by an exit arrow from state 0_D with a prospect set containing b ; remember that the guide sets of exit arrows do not back-propagate onto call arc chains; see the textbook for the guide set equations and their enabling conditions.

The other guide sets on the call arcs are simple and do not deserve any comment. The guide sets on the terminal shift arcs are immediate and here are omitted. The guide sets on the three bifurcation states 2_S , 0_C and 0_D of the *PCFG* are disjoint, thus grammar G is of type *ELL*(1). Actually this fact was already known by verifying the *ELL* condition on the pilot.

- (c) For completeness, here all three syntactic procedures S , C and D of the recursive descent analyzer are listed. First there are shown the plain non-optimized versions, obtained by testing each guide set case separately.

```

procedure  $S$ 
if  $cc \in \{ a \}$                                      // state 0
|    $cc = next$                                          // goto state 1
|   loop                                              // unconditional loop on state 1
|   |                                               // truncates by return or error
|   |   if  $cc \in \{ b \}$                              // state 1
|   |   |    $cc = next$                                // goto state 2
|   |   |   if  $cc \in \{ c, d \}$                      // state 2
|   |   |   |   call  $C$                              // go back to state 1
|   |   |   else if  $cc \in \{ b, \neg \}$              // state 2
|   |   |   |   return
|   |   |   else                                     // state 2
|   |   |   |   error
|   |   |   endif
|   |   else                                         // state 1
|   |   |   error
|   |   endif
|   end
| else                                               // state 0
| | error
| endif
end procedure

```

Procedure S is the axiomatic one. The main program is the usual one (see the textbook) and it is not shown here. Here are the two other procedures.

<pre> procedure <i>C</i> if $cc \in \{c\}$ // state 0 $cc = next$ // goto 1 if $cc \in \{b\}$ // state 1 $cc = next$ // goto 3 if $cc \in \{b\}$ // state 3 return else // state 3 error endif else // state 1 error endif else if $cc \in \{d\}$ // state 0 $cc = next$ // goto 2 if $cc \in \{a, b\}$ // state 2 call <i>D</i> // goto 3 if $cc \in \{b\}$ // state 3 return else // state 3 error endif else // state 2 error endif else // state 0 error endif end procedure </pre>	<pre> procedure <i>D</i> if $cc \in \{a\}$ // state 0 call <i>S</i> // goto 1 if $cc \in \{b\}$ // state 1 return else // state 1 error endif else if $cc \in \{b\}$ // state 0 return else // state 0 error endif end procedure </pre>
--	--

Of course there are several possible code optimizations, like grouping most error cases. Here is an optimized version of the axiomatic procedure *S*.

```

procedure S
if  $cc \in \{a\}$            // state 0
|    $cc = next$            // goto state 1
|   while  $cc \in \{b\}$  do   // conditional loop on state 1
|   |   // terminates by condition or truncates by return
|   |    $cc = next$        // goto state 2
|   |   if  $cc \in \{c, d\}$  // state 2
|   |   |   call C       // go back to state 1
|   |   else if  $cc \in \{b, \neg\}$  // state 2
|   |   |   return
|   |   endif
|   end
endif
error           // grouped error cases
end procedure

```

It is based on grouping the error cases and using a conditional loop, which

replaces the unconditional one and tests the guide set on the transition from state 1 to state 2. Furthermore, notice that the **while** loop is correct because if the return test on state 2 fails, then it is granted that $cc \neq b$ holds true and therefore the loop condition will fail as well, thus falling into the grouped error case at the procedure end.

The reader may implement the other optimized procedure versions by himself. However, here is a possible realization:

<pre> procedure <i>C</i> if $cc \in \{ c \}$ // state 0 $cc = next$ // goto 1 if $cc \in \{ b \}$ // state 1 $cc = next$ // goto 3 if $cc \in \{ b \}$ // state 3 return endif endif else if $cc \in \{ d \}$ // state 0 $cc = next$ // goto 2 if $cc \in \{ a, b \}$ // state 2 call <i>D</i> // goto 3 if $cc \in \{ b \}$ // state 3 return endif endif endif endif error // grouped error cases end procedure </pre>	<pre> procedure <i>D</i> if $cc \in \{ a \}$ // state 0 call <i>S</i> // goto 1 if $cc \in \{ b \}$ // state 1 return endif else if $cc \in \{ b \}$ // state 0 return endif error // grouped error cases end procedure </pre>
---	---

This possible optimization is based essentially on grouping the error cases. There may be more variants, of course.

4 Language Translation and Semantic Analysis 20%

1. Consider the following source grammar G_s (axiom S), for a language over the three-letter terminal alphabet $\{ a, b, c \}$ and the two-letter nonterminal alphabet $\{ S, A \}$:

$$G_s \left\{ \begin{array}{l} S \rightarrow A c S \\ S \rightarrow A \\ A \rightarrow a a A b b \\ A \rightarrow c b \end{array} \right.$$

Grammar G_s generates strings composed of a sequence of groups of letters separated by letters c , like for instance string:

$$a^4 c b^5 c a^2 c b^3$$

where each group consists of an *even* number (say $2k$ with $k \geq 0$) of letters a immediately followed by one letter c and then by $2k + 1$ letters b .

- (a) Write a syntactic translation scheme G_τ (or a translation grammar) that computes the following translation τ :

$$\tau(a^{2k_1} c b^{2k_1+1} c \dots c a^{2k_n} c b^{2k_n+1}) = b^{2k_1+1} d a^{2k_1} e \dots e b^{2k_n+1} d a^{2k_n} f$$

over the five-letter target alphabet $\{ a, b, d, e, f \}$, with $k_i \geq 0$ and $1 \leq i \leq n$.

Example:

$$\tau(a^4 c b^5 c a^2 c b^3) = b^5 d a^4 e b^3 d a^2 f$$

- (b) Draw the source and target syntax trees for the sample string $a^4 c b^5 c a^2 c b^3$ shown above.
- (c) Determine if the above translation τ can be defined by means of a finite-state translator T , and provide a suitable explanation for your answer.

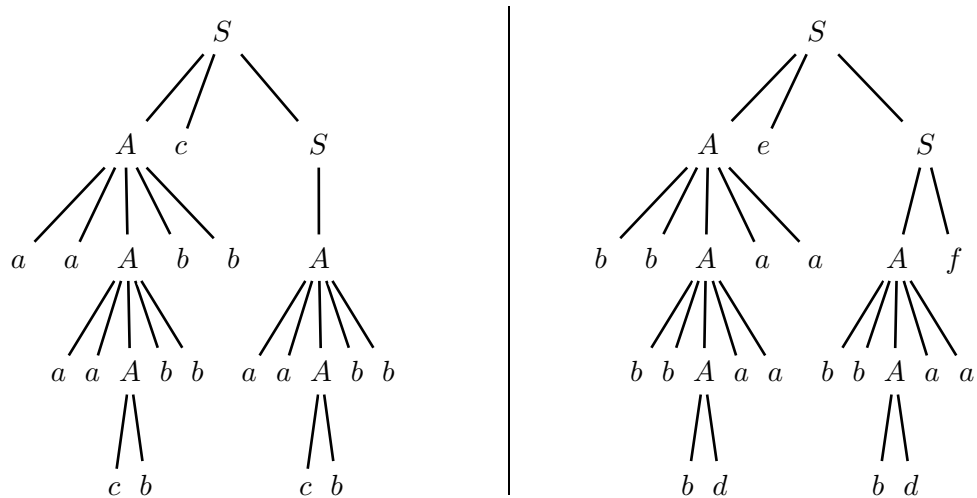
Solution

- (a) Here are a working translation scheme G_τ and aside the associated translation grammar (axiom S), of type *BNF*, based on the proposed source grammar G_s :

$$G_\tau \left\{ \begin{array}{ll|l} S \rightarrow A c S & S \rightarrow A e S & S \rightarrow A \frac{c}{e} S \\ S \rightarrow A & S \rightarrow A f & S \rightarrow A \frac{\varepsilon}{f} \\ A \rightarrow a a A b b & A \rightarrow b b A a a & A \rightarrow \frac{a}{b} \frac{a}{b} A \frac{b}{a} \frac{b}{a} \\ A \rightarrow c b & A \rightarrow b d & A \rightarrow \frac{\varepsilon}{b} \frac{c}{d} \frac{b}{\varepsilon} \end{array} \right.$$

Of course, the source and target grammars have exactly the same nonterminal structure, as it is mandated by the definition of syntactic translation. Notice that the final letter f is generated in the axiomatic rule $S \rightarrow A$ that terminates the right recursion of the alternative $S \rightarrow A \ c \ S$. Furthermore, a source letter c is translated into different target letters depending on the role it plays in the source string: into e if it plays as the separator of two consecutive groups a - b or into d if it plays as the centre of a group a - b . The rest of the translation scheme is quite simple, as the letters a and b of each group are just commuted.

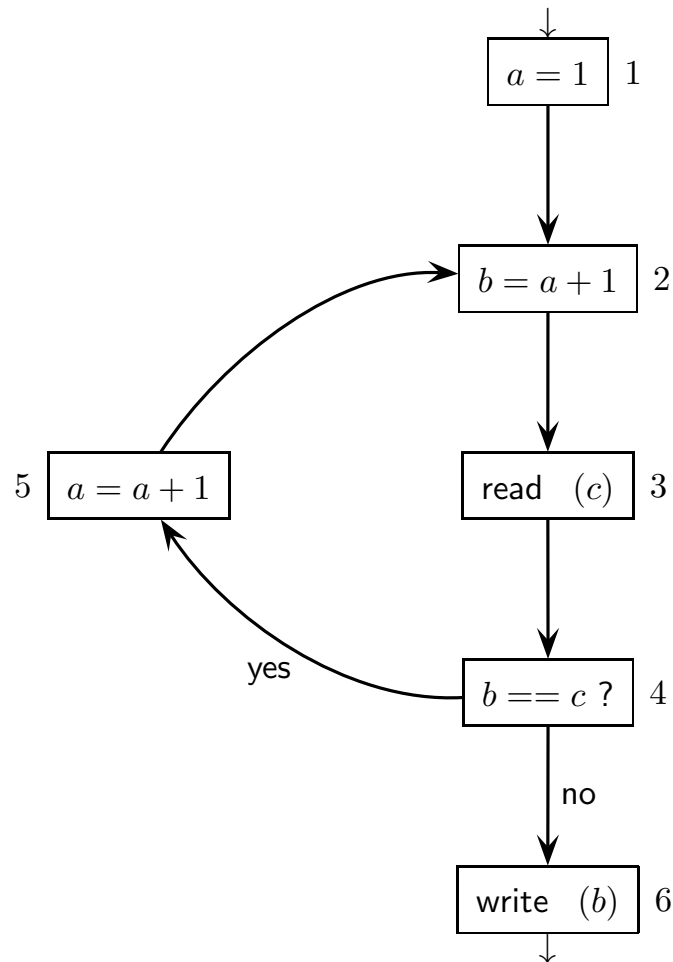
- (b) Here are the two requested source and target syntax trees, for the sample string:



- (c) It is impossible to define translation τ by means of a finite-state translator T , because the (hypothetical) underlying finite-state recognizer of T would be unable to count the matching letters a and b of each group a - b in the source string, hence it would fail to correctly check if an input string is valid, that is, if the string belongs to the source language. In fact, it is evident that each input group of matching letters a and b has the structure $a^n c b^{n+1}$ (with $n \geq 0$), that is, the group has a well known non-regular structure with two related exponents.

It would be relatively easy to design a finite-state translator that, given an input string *certainly belonging to the source language*, translates it into the correct matching string of the target language. Yet such a translator would be unable to correctly compute the translation if it were fed with a *generic* input string, because the string might be invalid and notwithstanding accepted.

2. Consider the following control flow graph (*CFG*) of a simple program:



Answer the following questions:

- Find the *live variables* of the given *CFG*, by means of the systematic method of the liveness flow equations: first write the equations, then iteratively solve them. Write the live variables on the *CFG* prepared on the next pages.
- Find the *reaching definitions* of the given *CFG*, by intuitively applying the definition of reaching definition. Write the reaching definitions on the *CFG* prepared on the next pages.
- (optional) Write the flow equations for the reaching definitions.

table of definitions and usages at the nodes for **LIVE VARIABLES**

<i>node</i>	<i>defined</i>	<i>node</i>	<i>used</i>
1		1	
2		2	
3		3	
4		4	
5		5	
6		6	

system of data-flow equations for **LIVE VARIABLES**

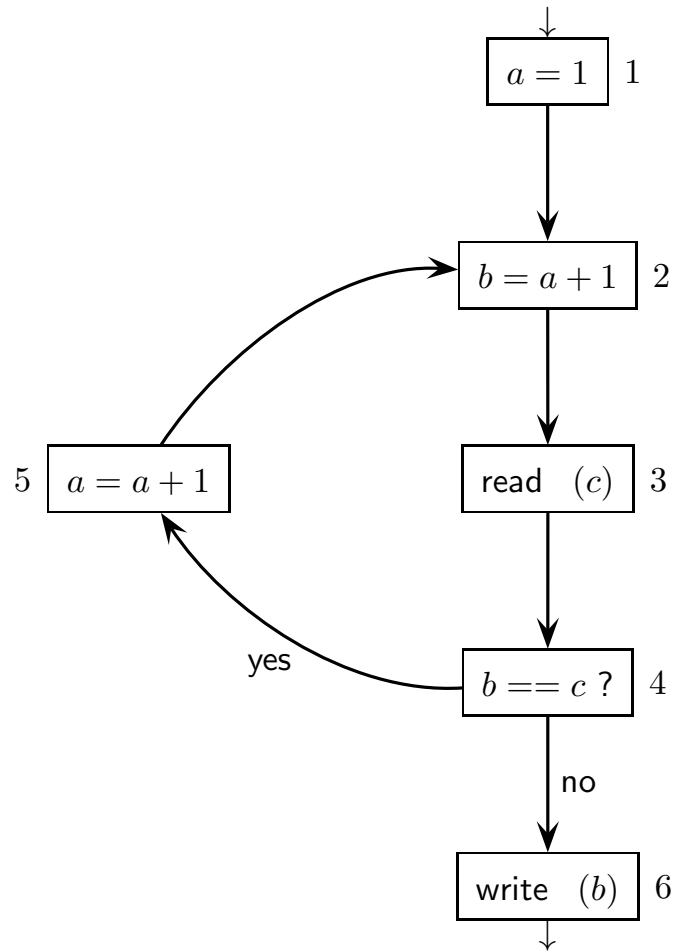
<i>node</i>	<i>in equations</i>	<i>out equations</i>
1		
2		
3		
4		
5		
6		

iterative solution table of the system of data-flow equations (**LIVE VAR.S**)
(the number of tables and columns is not significant)

	<i>initialization</i>		1		2		3		4		5		6	
#	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>
1														
2														
3														
4														
5														
6														

	7		8		9		10		11		12		13	
#	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>
1														
2														
3														
4														
5														
6														

please here write the **LIVE VARIABLES** obtained systematically



please here write the **REACHING DEFINITIONS** obtained intuitively

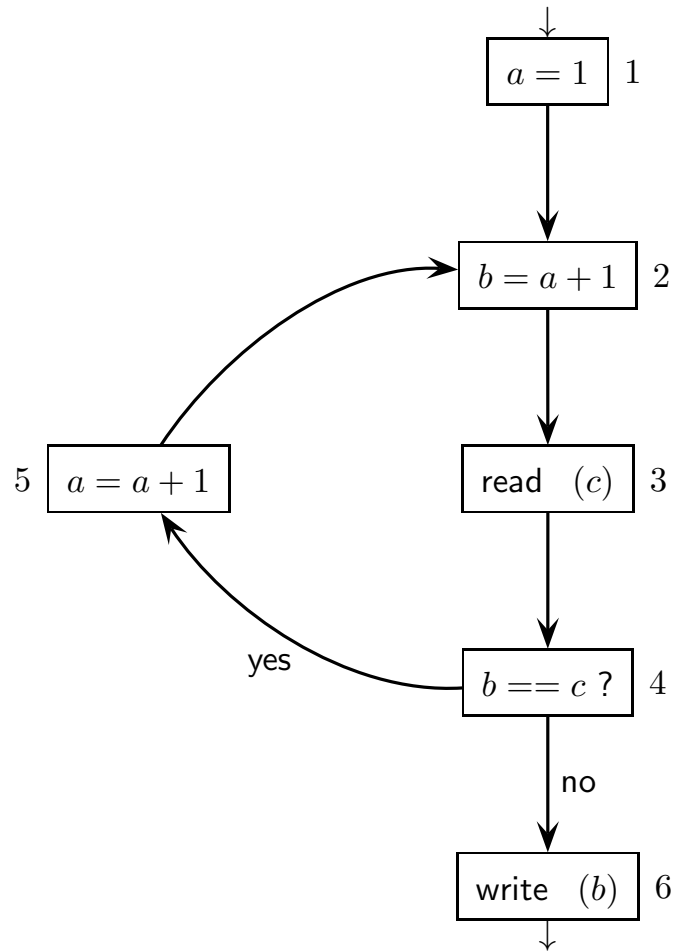


table of definitions and suppressions at the nodes
for **REACHING DEFINITIONS**

<i>node</i>	<i>defined</i>	<i>node</i>	<i>suppressed</i>
1		1	
2		2	
3		3	
4		4	
5		5	
6		6	

system of data-flow equations for **REACHING DEFINITIONS**

<i>node</i>	<i>in equations</i>	<i>out equations</i>
1		
2		
3		
4		
5		
6		

Solution

(a) Here is the systematic application of the flow equation method for live variables.

table of definitions and usages at the nodes for **LIVE VARIABLES**

<i>node</i>	<i>defined</i>	<i>node</i>	<i>used</i>
1	a	1	—
2	b	2	a
3	c	3	—
4	—	4	$b\ c$
5	a	5	a
6	—	6	b

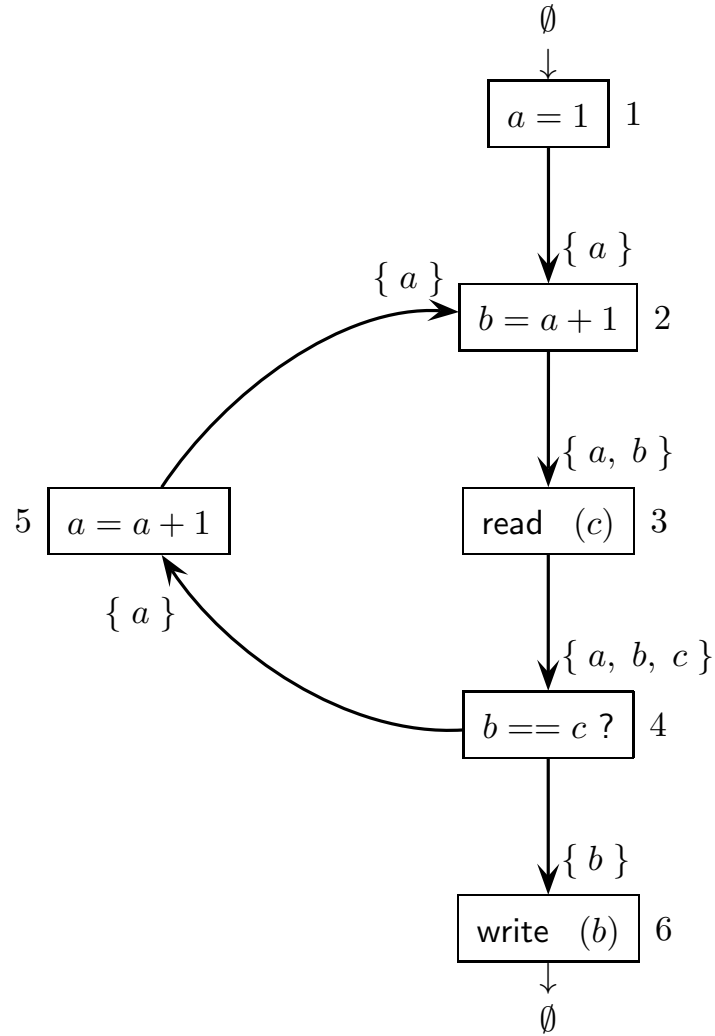
system of data-flow equations for **LIVE VARIABLES**

<i>node</i>	<i>in equations</i>	<i>out equations</i>
1	$in(1) = \emptyset$	$out(1) = in(2)$
2	$in(2) = \{ a \} \cup (out(2) - \{ b \})$	$out(2) = in(3)$
3	$in(3) = out(3) - \{ c \}$	$out(3) = in(4)$
4	$in(4) = \{ b, c \} \cup out(4)$	$out(4) = in(5) \cup in(6)$
5	$in(5) = \{ a \} \cup (out(5) - \{ a \})$	$out(5) = in(2)$
6	$in(6) = \{ b \} \cup out(6)$	$out(6) = \emptyset$

iterative solution table of the system of data-flow equations (**LIVE VAR.S**)

	<i>initialization</i>		1		2		3		4		5	
#	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>	<i>out</i>	<i>in</i>
1	$\emptyset$	—	a	$\emptyset$	a	$\emptyset$	a	$\emptyset$				
2	$\emptyset$	a	—	a	b	a	$a\ b$	a				
3	$\emptyset$	—	$b\ c$	b	$a\ b\ c$	$a\ b$	$a\ b\ c$	$a\ b$				
4	$\emptyset$	$b\ c$	$a\ b$	$a\ b\ c$	$a\ b$	$a\ b\ c$	$a\ b$	$a\ b\ c$				
5	$\emptyset$	a	a	a	a	a	a	a				
6	$\emptyset$	b	$\emptyset$	b	$\emptyset$	b	$\emptyset$	b				

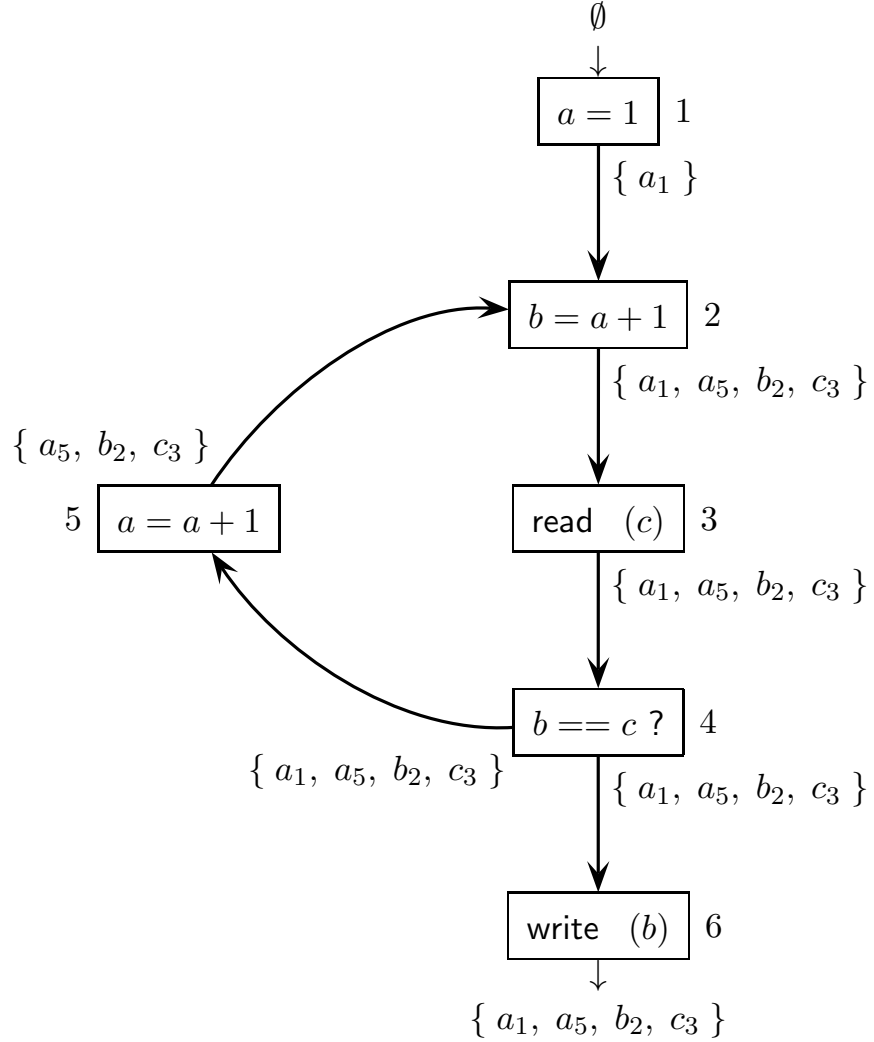
LIVE VARIABLES obtained systematically



The iterative solution procedure converges in three steps. Here we choose to show the variables that are LIVE at the INPUT of each node (we visually express this choice by listing the variables next to the arc tips), including the initial one, which does not have any. Of course, the arcs going into the same node bear the same variables. Since the program does not have any output parameters, there are not any variables live at the OUTPUT of the final node. Alternatively, one might choose to represent the variables live at the node outputs.

(b) Here is the intuitive solution of the reaching definitions.

REACHING DEFINITIONS obtained intuitively



Here we choose to show the definitions that are **REACHING** the **OUTPUT** of each node (we visually express this choice by listing the definitions next to the arc origins), including the final one. Of course, the arcs coming from the same node bear the same definitions. Since all the variables are internally defined before being used and the program does not have any input parameters, there are not any definitions reaching the **INPUT** of the initial node. Alternatively, one might choose to represent the definitions reaching the node inputs.

- (c) Here is the systematic application of the flow equation method for reaching definitions.

table of definitions and suppressions at the nodes
for **REACHING DEFINITIONS**

<i>node</i>	<i>defined</i>	<i>node</i>	<i>suppressed</i>
1	a_1	1	a_5
2	b_2	2	—
3	c_3	3	—
4	—	4	—
5	a_5	5	a_1
6	—	6	—

Basically, the definitions a_1 and a_5 of variable a suppress each other. There are not any other suppression cases. The rest (definitions) is quite obvious and similar to the live variables, with the refinement of indexing the definitions.

system of data-flow equations for **REACHING DEFINITIONS**

<i>node</i>	<i>in equations</i>	<i>out equations</i>
1	$in(1) = \emptyset$	$out(1) = \{ a_1 \} \cup (in(1) - \{ a_5 \})$
2	$in(2) = out(1) \cup out(5)$	$out(2) = \{ b_2 \} \cup in(2)$
3	$in(3) = out(2)$	$out(3) = \{ c_3 \} \cup in(3)$
4	$in(4) = out(3)$	$out(4) = in(4)$
5	$in(5) = out(4)$	$out(5) = \{ a_5 \} \cup (in(5) - \{ a_1 \})$
6	$in(6) = out(4)$	$out(6) = in(6)$

The reader may verify by himself that the previous reaching definitions obtained intuitively satisfy this equation set of reaching definitions. Alternatively, he may wish to iteratively solve the equation set and verify that the systematic solution obtained is identical to the intuitive one.